

First-order Logic Basics

Video Lesson

Duration: 00:23:26

Slide Deck

M09S01 (<https://canvas.tamu.edu/courses/430127/files/92570673?wrap=1>) 

Lesson Transcript

In this lecture, we will look into first-order logic basics. In the series of lectures in this module, we will look at first-order logic and automated theorem proving within first-order logic.

Predicate Calculus (First-Order Logic)



Treatment of predicate calculus in this lecture closely follows: Chang, C.-L., and Lee, R. C.-T., "Symbolic Logic and Mechanical Theorem Proving", Academic Press, London, 1973. Substitution notation follows Russell & Norvig.

First-order Logic is also known as Predicate Calculus.

The treatment of predicate calculus in this lecture closely follows Chang and Lee, the book titled "Symbolic Logic and Mechanical Theorem Proving."

Predicate Calculus (First-Order Logic)



Propositional logic does not allow us to perform any reasoning based on the use of general rules, so its usefulness is limited. Predicate Calculus generalizes Propositional Calculus to allow the expression and use of **general rules**.

- objects
- relations
- properties
- functions

As we have seen in the previous videos, propositional logic does not allow us to perform any reasoning based on the use of general rules, so it is limited in its usefulness.

Predicate Calculus or first-order logic generalizes propositional calculus to allow the expression and use of general rules.

These rules allow us to talk about objects, the relationship between these objects, the properties of these objects, and how objects turn into other objects through functions.

New Concepts Introduced in Predicate Calculus

- **terms** : objects in the domain, and how things get transformed (**functions**)
- **predicates** : properties of objects or relations (certain properties are **True** or **False**)
- **quantifiers** : express properties of large set of objects without enumerating



There are several new concepts introduced in first-order logic. These are terms, predicates, and quantifiers.

Terms are objects in the domain and how things get transformed into each other.

Predicates are properties of objects or their relations where certain properties can be either true or false.

Quantifiers allow us to express the properties of large sets of objects without enumerating them explicitly.

Terms in Predicate Calculus

A **Term** is:

- constant: $a, b, c, \dots$
- variable: $x, y, z, \dots$
- $f(t_1, \dots, t_n)$, where f is a function symbol and $t_1, t_2, \dots, t_n$ are terms.

Terms refer to objects in a domain.

A term in first-order logic is either a constant, a variable, or a function.

Constants are named entities in the domain, whereas variables can hold any of these constants or other variables, and functions take in as arguments other terms. Again, these terms could be constants, variables, or even other functions and return a single value, which is another term.

Predicate Calculus Constructs

- Variables: $x, y, z, \dots$
- Constants: John, Mary, 3
- Functions: $f(x), g(y), h(z), \text{father}(\text{John}), \dots$
 - maps **term(s)** to a **term**
- Predicates: $P(x, y), \text{GREATER}(x, 3), \text{LOVE}(\text{father}(\text{John}), \text{John})$
 - function whose value is **True** or **False**
- Quantifiers: $\forall$ (for all), $\exists$ (there exists)

Here are some examples of terms. Variables: x, y, z . Constants: John, Mary, 3. And functions $f(x), g(y), h(z), \text{father}(\text{John})$. In this case, $\text{father}(\text{John})$ would return another person in the domain who is the actual father of John. So, functions map terms to other terms.

As I mentioned earlier, there are three new constructs in first-order logic. One is the term, the second is a predicate, and the final one is the quantifiers. Predicates take the form like this, $P(x, y)$, where x and y are terms.

And $\text{GREATER}(x, 3)$. It means x , whatever that entity is, the term is greater than 3. And if that is the case, then this predicate would have a value of **True**.

Here is another one. $\text{LOVE}(\text{father}(\text{John}), \text{John})$. So, if the function $\text{father}(\text{John})$ returns, the actual father of John, say John Sr. Then, if John Sr loves John Jr., then this predicate would be **True**. Otherwise, this would return **False**.

Finally, we have quantifiers. This upside-down A, $\forall$, means "for all", and this flipped E, $\exists$, means "there exists", and these quantifiers are always associated with some kind of variable.

Mortality Revisited

Propositional logic

- P : All men are mortal
- Q : Socrates is a man
- R : Socrates is mortal

First-order logic

- P : All men are mortal $\forall x \quad MAN(x) \rightarrow MORTAL(x)$
- Q : Socrates is a man $MAN(\text{Socrates})$
- R : Socrates is mortal $MORTAL(\text{Socrates})$

Let us look at the mortality example we saw earlier in propositional logic.

In propositional logic, these three sentences would each reduce to one atom. So, "All men are mortal" would be a single atom, P . "Socrates is a man" would be, again, a single atom, Q . And "Socrates is mortal" would be another atom, R . So, it is hard to do any kind of reasoning given just P , Q , and R .

In contrast, in first-order logic, each of these three sentences can be broken down into smaller components. For example, the first sentence, "all men are mortal", can be written: $\forall x \quad MAN(x) \leftrightarrow MORTAL(x)$. As I mentioned, this variable x is associated with this universal quantifier upside down A.

Next, "Socrates is a man" would be turned into $MAN(\text{Socrates})$, where Socrates is a term and also a constant. So, this says that the predicate MAN has a **True** value when the argument is Socrates, and that describes the sentence "Socrates is a man".

Likewise, the last sentence, "Socrates is mortal", can be written as $MORTAL(\text{Socrates})$.

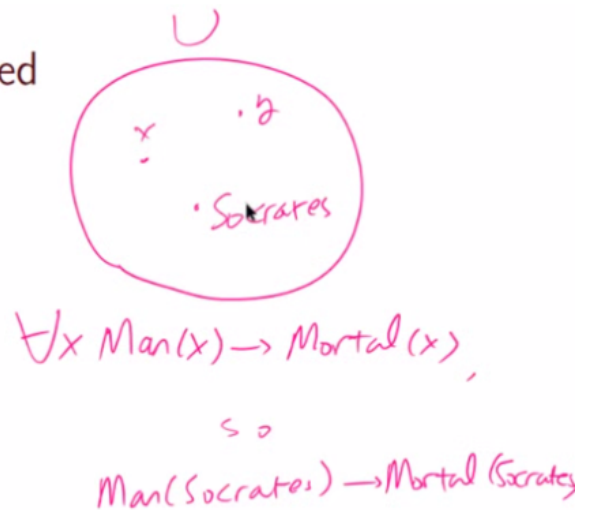
Mortality Revisited

Propositional logic

- P : All men are mortal
- Q : Socrates is a man
- R : Socrates is mortal

First-order logic

- P : All men are mortal $\forall x \text{ } MAN(x) \rightarrow MORTAL(x)$
- Q : Socrates is a man $MAN(\text{Socrates})$
- R : Socrates is mortal $MORTAL(\text{Socrates})$



Let us think about the population of people in this domain. Because $\forall x$ in this collection of people, $MAN(x)$ implies $MORTAL(x)$, and Socrates is part of this population, we can deduce that $MAN(\text{Socrates}) \leftrightarrow MORTAL(\text{Socrates})$.

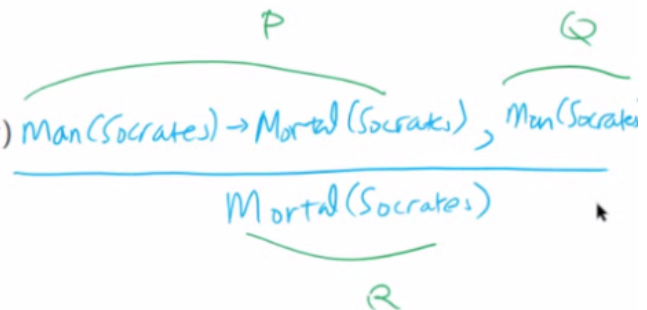
Mortality Revisited

Propositional logic

- P : All men are mortal
- Q : Socrates is a man
- R : Socrates is mortal

First-order logic

- P : All men are mortal $\forall x \text{ } MAN(x) \rightarrow MORTAL(x)$
- Q : Socrates is a man $MAN(\text{Socrates})$
- R : Socrates is mortal $MORTAL(\text{Socrates})$



Now, if we arrange P , Q , and R in the form shown on the right, we can identify this as modus ponens. The $MAN(\text{Socrates}) \leftrightarrow MORTAL(\text{Socrates})$, and the premise part of the implication $MAN(\text{Socrates})$, is also given by Q , given that we can deduce $MORTAL(\text{Socrates})$, which is the last sentence.

A Formal Definition

Well Formed Formula (or Sentence):

WFF	= Atomic-Formula WFF Connective WFF Quantifier Variable, WFF $\neg$ -WFF (WFF)
Atomic-Formula	= Predicate(Term, ...) Term = Term
Term	= Function(Term, ...) Constant Variable
Connective	= $\rightarrow$ $\wedge$ $\vee$ $\leftrightarrow$
Quantifier	= $\forall$ $\exists$
Constant	= A X_1 John ...
Variable	= a x s ...
Predicate	= Before HasColor Raining ...
Function	= Mother LocationOf ...

As in propositional logic, we can define the well-formed formula in first-order logic using this recursive rule.

A well-formed formula is either an atomic formula or a well-formed formula connected with another well-formed formula using one of these logical connectives, or it is a sentence consisting of a quantifier and a variable and a well-formed formula where this variable appears or is a negation of a well-formed formula or parentheses applied to a well-formed formula.

In propositional logic, the atom was something that could have a true or false value. The atomic formula in first-order logic is the same. It is something that can take a true or false value. Naturally, predicates are atomic formulas because when we apply certain arguments, then this predicate could be either **True** or **False**.

In addition, we have an equivalence check. If a term is the same as another term, then this returns a true value. Otherwise, it returns a false value. Here, since terms can be constants, variables, or functions, we can compare any of these three types.

Next, terms can be either constant or variable or a function of different terms, which we saw earlier. And connectives are the same as in propositional logic. We have implication, conjunction, disjunction, and equivalence. And the quantifier symbol here could be either the universal quantifier $\forall$, the upside-down A, or the existential quantifier $\exists$, the flipped E.

Here are some examples of constants: *a*, *b*, *c*, *d*, X_1 , X_2 , John, Mary, 3, and so on.

Variables are usually written in lowercase. So, *a*, *x*, *y*, *s*, *z*, et cetera. Again, these variables are always associated with a specific type of quantifier, either universal or existential.

Predicates are, again, certain symbols that would indicate the property relation, such as *Before*,

HasColor of some color, is it raining or not, and so on.

The function maps one term to another term. Who is the mother of a certain person? Or what is the location of a particular object?

Functions vs. Predicates

- Functions: returns a single object (term); relations
 - *FatherOf*(*GeorgeJr*) = *GeorgeSr*,
 - *DistanceBetween*(*MilkyWay*, *Andromeda*) = 2-million light years, ...
- Predicates: returns a truth value; properties
 - *IsFather*(*GeorgeSr*, *GeorgeJr*) = **True**
 - *HeavierThan*(*Earth*, *Sun*) = **False**

Must disambiguate: *Brother*(*x*, *y*) could be

- *AreBrothers*(*x*, *y*) :predicate, or
- *BrotherOf*(*x*, *y*) :function, i.e. a common brother of *x* and *y*.

It is important to distinguish functions and predicates because they have very similar forms. For example, *Fatherof*(*GeorgeJr*) is the father of George Junior, i.e., George Senior. It has a symbolic name and parentheses, where we can specify the arguments.

Functions return a single object, which is a term, whereas predicates return a truth value of **True** or **False**.

So, in this example, when we plug in George Junior in the function *FatherOf*(), the return value would be George Senior. Likewise, for this function *DistanceBetween*(), we plugin, MilkyWay and Andromeda, then the result would be a term, 2 million light years. That would be a constant.

Predicates, on the other hand, would return a **True** or **False** value. So, this predicate *IsFather*(), when given these two arguments, George Senior and George Junior, will return the value of **True**.

Of course, the ordering of the arguments is very important. If this was the given context and we plugged in George Junior ahead of George Senior, then this predicate should return **False**.

The next predicate is *HeavierThan*(). Is the Earth heavier than the Sun? Of course, it is not true, so this should return **False**. But if we plug in *HeavierThan*(*Sun*, *Earth*), it should return **true**.

There are cases where there could be ambiguities. For example, when we write *Brother*(*x*, *y*), is it a function or a predicate? To make that distinction, it is better to be more explicit. So, instead of just writing *Brother*(*x*, *y*), we should write *AreBrothers*(*x*, *y*). Then this should return either *True* or

False if x and y were brothers or not.

Another interpretation of $Brother(x, y)$ is we want to know whether another person, z is a mutual brother of x and y . So, in that case, we want to write it more explicitly as $BrotherOf(x, y)$.

Quantifiers

$\forall \text{var } wff$

- Universal quantifier $\forall$:
 - Every *Skunk* is *Stinky* : translates into $\forall x \text{ Skunk}(x) \rightarrow \text{Stinky}(x)$
 - note that the main connective is $\rightarrow$

$\exists \text{var, } wff$

- Existential quantifier $\exists$:
 - There exists a *Cat* that is *White* : translates into $\exists x \text{ Cat}(x) \wedge \text{White}(x)$
 - Same as: Some *Cat* is *White*
 - note that the main connective is $\wedge$

Next, let us look at Quantifiers. The first one is the universal quantifier, $\forall$ followed by a variable and then by a well-formed formula.

Let us consider this sentence, "Every skunk is stinky". This translates to $\forall x \text{ Skunk}(x) \leftrightarrow \text{Stinky}(x)$. The important thing to remember here is that for this, the main connective should be the implication.

Next is an existential quantifier, followed by a variable and a well-formed formula. So, here is another example sentence, "There exists a cat that is white". This translates into $\exists x$ such that $\text{Cat}(x) \wedge \text{White}(x)$. Again, in this case, the main connective is a conjunction.

We will see in the next slide what happens when we mix up these two main connectives.

Common Mistakes With Quantifiers

All skunks are stinky:

- Correct: $\forall x \text{ Skunk}(x) \rightarrow \text{Stinky}(x)$
- Wrong: $\forall x \text{ Skunk}(x) \wedge \text{Stinky}(x)$
 - this means: everything is a skunk **and** it is stinky.

Some cats are white:

- Correct: $\exists x \text{ Cat}(x) \wedge \text{White}(x)$
- Wrong: $\exists x \text{ Cat}(x) \rightarrow \text{White}(x)$
 - this is true if there is something that is **not** a cat!

A common mistake with quantifiers is to use the incorrect connective.

Let us again consider this statement "All skunks are stinky". The correct expression is $\forall x \text{ Skunk}(x) \leftrightarrow \text{Stinky}(x)$. However, if instead of implication, we wrote conjunction, it becomes $\forall x \text{ Skunk}(x) \wedge \text{Stinky}(x)$. This means that whoever is in this domain, everything is a skunk, and everything is stinky.

Next, let us consider the existential quantifier. So, here is the sentence again, "some cats are white" or "there exists the cat that is white". So, the correct rendering of this is $\exists x$, such that $\text{Cat}(x) \wedge \text{White}(x)$. So again, the correct connective is a conjunction. However, if we wrote implication instead of conjunction, then the meaning is entirely different:

There exists a cat x , then x is white.

Common Mistakes With Quantifiers

All skunks are stinky:

- Correct: $\forall x \text{ Skunk}(x) \rightarrow \text{Stinky}(x)$
- Wrong: $\forall x \text{ Skunk}(x) \wedge \text{Stinky}(x)$
 - this means: everything is a skunk **and** it is stinky.

Some cats are white:

- Correct: $\exists x \text{ Cat}(x) \wedge \text{White}(x)$
- Wrong: $\exists x \text{ Cat}(x) \rightarrow \text{White}(x)$
 - this is true if there is something that is **not** a cat!

$x = \text{Fido the Dog}$
then
 $\text{Cat}(\text{Fido}) = \text{false}$

Consider this case. Let us say $x = \text{"Fido the dog"}$. Then when we plug Fido into this predicate cat, it will give us **False**. So, this premise part of the implication will be **False**, and x equals Fido is the example. Then this entire sentence becomes **True**. So, the meaning is very different from there exists x , such that x is a cat and x is white.

So, that means that there is some white cat.

Properties of Quantifiers

- $\forall x \forall y = \forall y \forall x$
- $\exists x \exists y = \exists y \exists x$
- $\forall x \exists y \neq \exists y \forall x$
 $\exists x \forall y \text{ Loves}(x, y)$ vs.
 $\forall y \exists x \text{ Loves}(x, y)$
- quantifiers can be translated using each other:
 $\forall x \text{ Likes}(x, \text{Coffee})$
 $\exists x \text{ Likes}(x, \text{Broccoli})$
 $\neg \exists x \neg \text{Likes}(x, \text{Coffee})$
 $\neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

Here are some important properties of these quantifiers. When we have $\forall x \forall y$, that is, these two

quantifiers are of the same type, then we can switch the ordering to $\forall y \forall x$. And then the same is true for existential quantifiers. $\exists x \exists y = \exists y \exists x$.

But when we have these two quantifiers of mixed type, we cannot change the order. $\forall x \exists y$ is not the same as $\exists y \forall x$.

Properties of Quantifiers

- $\forall x \forall y = \forall y \forall x$
- $\exists x \exists y = \exists y \exists x$
- $\forall x \exists y \neq \exists y \forall x$
 $\exists x \forall y \neg \text{Loves}(x, y)$ vs.
 $\forall y \exists x \text{ Loves}(x, y)$
- quantifiers can be translated using each other:
 $\forall x \text{ Likes}(x, \text{Coffee})$
 $\exists x \text{ Likes}(x, \text{Broccoli})$



$\neg \exists x \neg \text{Likes}(x, \text{Coffee})$
 $\neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

For example, let us consider these two sentences where the ordering of the existential and the universal quantifiers are flipped.

So, the first one is $\exists x \forall y$, x loves y . To illustrate that, let us look at this figure. So, here is x in the middle; there is someone x who loves everyone else, including himself or herself. So, this is an example where this is **True**.

Whereas a second case $\forall y \exists x$ such that x loves y . Here, y is on the receiving end of the love. This means that for every arbitrary person y , there is someone x who loves this person y . Because if someone loves himself or herself, that will also count.

So, as we can see, these two have a very different meaning.

Properties of Quantifiers

- $\forall x \forall y = \forall y \forall x$
- $\exists x \exists y = \exists y \exists x$
- $\forall x \exists y \neq \exists y \forall x$
 $\exists x \forall y \text{ Loves}(x, y)$ vs.
 $\forall y \exists x \text{ Loves}(x, y)$
- quantifiers can be translated using each other:
 $\forall x \text{ Likes}(x, \text{Coffee})$
 $\exists x \text{ Likes}(x, \text{Broccoli})$
 $\neg \exists x \neg \text{Likes}(x, \text{Coffee})$
 $\neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

We can also translate one quantifier to the other type. For example, $\forall x$, x likes coffee can be translated into there does not exist x , such that x dislikes coffee.

$\neg \exists x \neg \text{Likes}(x, \text{Coffee})$

We can do the same for the existential quantifier. There exists x , such that x likes broccoli can be translated into not for all x , x dislikes broccoli.

$\neg \forall x \neg \text{Likes}(x, \text{Broccoli})$

Semantics of Predicate Calculus

Formulas are true with respect to a **model** and an **interpretation**.

Models contain **objects** and **relations**:

- objects: constants
- relations and properties: predicates
- functional relations: functions

An atomic formula $\text{Predicate}(\text{term}_1, \text{term}_2, \dots, \text{term}_n)$ is true iff the **objects** referred to by $\text{term}_1, \text{term}_2, \dots, \text{term}_n$ are in the **relation** referred to by Predicate .

I

As we have seen in propositional logic, semantics in logic means whether a particular sentence is **True** or **False**. For first-order logic, to check if a formula is true or not, we have to determine what the model is and what the interpretation is. Models contain objects and relations, where objects are constants, relations, and properties are predicates, and functional relationships between

objects are functions.

An atomic formula, which is a predicate or an equivalence test, will return true or false when certain properties hold across these terms in the argument.

Example: Howling Hounds

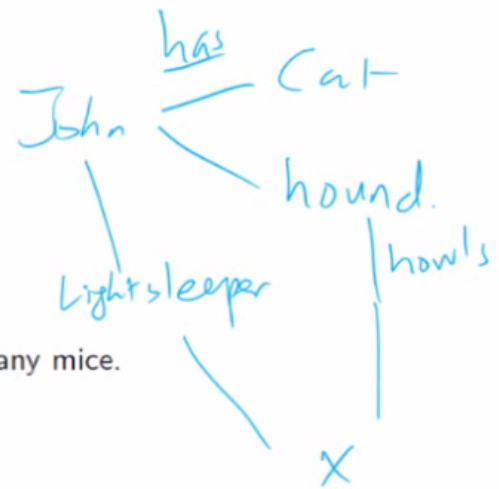
- ① All hounds howl at night.
- ② Anyone who has any cats will not have any mice.
- ③ Light sleepers do not have anything which howls at night.
- ④ John has either a cat or a hound.
- ⑤ **Prove:** If John is a light sleeper, then John^I does not have any mice.

To motivate what follows, let us look at the following theorem in first-order logic. This theorem is called Howling Hounds. It consists of four premises. And the fifth one is the statement that we want to prove. The first sentence is, "All hounds howl at night". The second is, "Anyone who has any cats will not have any mice", so that kind of makes sense.

Next, "light sleepers do not have anything which howls at night". Next, "John has either a cat or a hound". Finally, based on all of these four sentences, we want to prove "If John is a light sleeper, then John does not have any mice".

Example: Howling Hounds

- 1 All hounds howl at night.
- 2 Anyone who has any cats will not have any mice.
- 3 Light sleepers do not have anything which howls at night.
- 4 John has either a cat or a hound.
- 5 **Prove:** If John is a light sleeper, then John does not have any mice.



Here is a rough sketch of what is going on in this theorem. John either has a cat or a hound, but hounds howl at night, but light sleepers do not have anything that howls at night. So, John cannot have a hound.

Due to that, John certainly has a cat.

Then finally, there is the last critical part. Whoever has a cat will not have any mice, and because we have shown that John has a cat, John should not have mice.

Since that is shown, we have proven this last sentence.

Example: Howling Hounds (cont'd)

- 1 $\forall x (HOUND(x) \rightarrow HOWL(x))$
- 2 $\forall x \forall y ((HAVE(x, y) \wedge CAT(y)) \rightarrow \neg \exists z (HAVE(x, z) \wedge MOUSE(z)))$
- 3 $\forall x (LS(x) \rightarrow \neg \exists y (HAVE(x, y) \wedge HOWL(y)))$
- 4 $\exists x (HAVE(John, x) \wedge (CAT(x) \vee HOUND(x)))$
- 5 **Prove:** $LS(John) \rightarrow \neg \exists x (HAVE(John, x) \wedge Mouse(x))$

To prove this theorem automatically using an algorithm, we need first to turn it into first-order logic. So, these five sentences become,

1. For all x hound x implies howl x
 2. Then, For all x , for all y , x has y and $CAT(y)$, there does not exist a z such that the same x here who has the cat does not have this mouse, z .
 3. Next, For all x , if x is a light sleeper, then there does not exist something y , such that this x , who is a light sleeper, has this y , and y howls.
 4. Next, there exists x , such that John has x , and x is either a cat or a hound.
 5. And finally, we want to prove John is a light sleeper. Then there exists something x , so John has x , and x is a mouse. But since there is a negation, it means that John does not have this mouse x , or there does not exist something x . There is a mouse, and John has it.
-

Canonical Forms of Predicate Calculus

- ① Prenex Normal Form: arranged all quantifiers at the front of the formula : use De Morgan's rules (p. 193)
- ② Convert the non-quantifier part (called the **matrix**) into Conjunctive Normal Form
- ③ Skolemization: eliminate existential quantifiers by introducing **Skolem constants** or **Skolem functions**.

Result:

$\forall x_1 \forall x_2 \dots \forall x_n (\text{CNF})$

I

As in propositional logic, we have to turn in the first-order logic sentences into a form that is easy for machines to handle. These are called mechanical forms in first-order logic.

This process is a little bit more complex than propositional logic, where the only thing we need to do is to turn the expression into conjunctive normal form. In first-order logic, because we have quantifiers, it becomes more complex. So, first, we turn the first-order logic sentence into something called the Prenex Normal Form.

Here, the task is to arrange all of the quantifiers so that they appear at the very beginning of the sentence.

Next, we convert the non-quantifier part, which is technically called the matrix, into Conjunctive Normal Form. And finally, we can also eliminate all of these existential quantifiers through a process called Skolemization.

We will look at the detailed steps for each of these in the following lectures. As a result of turning the first-order logic sentence into this canonical form, we should get something that looks like this:

$\forall x_1 \forall x_2 \dots \forall x_n$, and the right-hand side would contain the conjunctive normal form.

Key Points

- predicate calculus basics
- quantifier properties, and common mistakes
- translating English into predicate calculus
- canonical forms for predicate calculus: basics

This concludes the introduction to first-order logic. In the following lecture, we look at a more concrete example of how the truth value can be determined.